What survives when a paper leaves the lab: re-testing 21 robot-learning papers on a €1,350 rig

Ravindu Somawansa
Shisso Robotics
ravindu.somawansa@shissorobotics.com

Abstract.

- Most robot-learning papers are tested in simulation or on rigs above $20k: 15 of our 21 papers; 2 under $5k.
- Question: do their claims still hold on a cheap rig?
- We re-tested 37 claims from 21 papers on an SO-101 arm and a Jetson Orin NX 16 GB (€1,350 of parts), with $42.30 of cloud GPU.
- Data: our own teleoperated demonstrations, our own policy rollouts, and a public set of SO-101 rollouts.
- Two rules: the bar is fixed before the test, from the paper and its code only; the cheapest baseline is run first, on the same data.
- Result: 12 claims hold, 18 fail, 7 are not settled.
- Three failure classes: too slow for the edge GPU; judges that lose to episode length; latents that lose to the commanded joints.
- Advice: start any re-test from the cheapest baseline.
- Code, data and every number are public.

Code and rows: `github.com/ravediamond/paper-register-reproducibility` **Reproduce:** `make check`. Every number carries its source file and protocol commit.

1 Introduction

- 15 of 21 papers shown in simulation or on a rig above $20k; 2 on a rig under $5k like ours.
- A small team has to guess, or re-test every claim itself, which is slow and costly.
- This paper: re-test the claims, one at a time, on one cheap rig.
- Example, adaptive chunking:
  - idea: choose the chunk length from the entropy of sampled chunks; long in transport, short at the grasp
  - one decision needs 20 sampled chunks → 4,751 ms on the Orin, budget 298 ms → fail
  - a fail on our rig, not on their hardware
- Overall: 12 of 37 hold, 18 fail.
- Several failures are not near misses: the method loses to a baseline with no learning → three classes, Section 5.

2 Rig and data

- Rig: SO-101 arm (5 joints + gripper), top and wrist cameras, Jetson Orin NX 16 GB for inference; training on rented cloud GPUs.
- Teleoperated demonstrations (`so101_bowls_all_trimmed`): 182 episodes, 61,462 frames at 30 fps (about 34 minutes); two short bowl tasks (one bowl into another); split 157 train / 25 validation episodes. Used to train and test the world models and VLAs.
- Policy rollouts (`so101-bowls-policy-rollouts`): 69 episodes of our trained policies on the arm, 44 successes and 25 failures, labelled by the operator. Used for the failure monitors.
- ArmnetBench [14]: 2,099 public labelled SO-101 rollouts from 7 policies. Used for the reward models and judges.
- Timing: 20 recorded episodes, 254 chunk boundaries. Used for chunking and latency.
- Limits of the data:
  - short tasks only, about 11 s per episode; no long-horizon task (several steps, memory) → we say nothing about long horizon
  - one task family, one arm, one room
  - outcome labels from one operator, not blind

Table 1: The cheapest baseline for each kind of method, run on the same data before the method.

method	cheapest baseline	why it is a fair test
state predictor	copy-last: the next state is the current one	costs nothing
learned latent	the commanded joint values	the arm's state is already in them
success or failure judge	episode length	on our rig it already says a lot about the outcome

3 How we test

- Rule 1: the bar is written from the paper and its code only, before any result; a bar written after the result drifts toward it.
- Rule 2: the cheapest baseline first, on the same data (Table 1); if the method loses to it, the bar barely matters.
- Verdicts: holds (bar met), fails (bar missed), not proven (test ran, cannot decide), not tested (planned).
- A fail can be our fault, so we run a control. Example, TurboVLA (our mistake):
  - we thought the architecture made our port of the policy ignore its instruction
  - bar: swapping the task moves the output by $\geq 5°$ → our port is the bug; by $< 1°$ → the data
  - upstream's own trainer on our data: 0.020°; their released LIBERO checkpoint on our rig: 5.5°
  - → our data, not their architecture; we withdrew the claim, and it is not counted here

4 Results

- Figure 1 and Table 2.
- Claims: 12 hold, 18 fail, 3 not proven, 4 not tested.
- Papers: 4 of 21 reproduce, 13 fail, 3 not proven, 1 not tested.
- Every paper, with its claim, bar and our result: appendix.

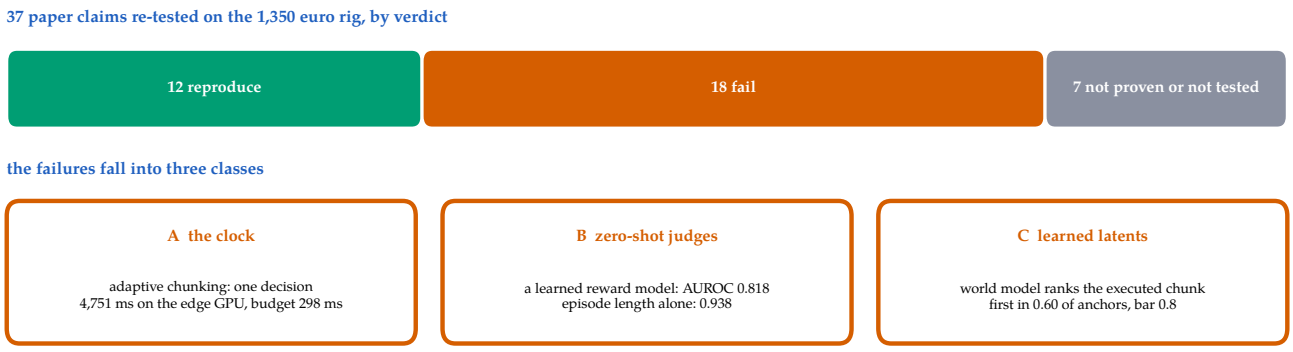


Figure 1: what survived when the papers left the lab: of the 37 paper claims, 12 reproduce and 18 fail; the failures fall into three classes: the clock, zero-shot judges against a trivial signal, and learned latents against the commanded joints.

Table 2: Verdicts by paper and by claim.

verdict	papers	claims
reproduces	4	12
fails	13	18
not proven	3	3
not tested	1	4

5 Three failure classes

- A, the clock: adaptive chunking, 4,751 ms for one decision against 298 ms; may work on a faster GPU; easiest to avoid: time the method on your device first.
- B, judges (a model that says success, failure or progress): a 4B reward model, zero-shot on ArmnetBench, AUROC 0.818; episode length alone 0.938.
- C, latents (a world model's vector computed from the camera): ranks the executed chunk first in 0.60 of anchors, bar 0.8; a baseline with the actions only: 0.68–0.90; the commanded joints already hold the arm's state, so the latent must add the objects, and it does not.
- The classes are our proposal, from the author's reading of the 18 failed claims; a second reader is needed.
- Order to test a paper on a cheap rig: time it on the device → the cheapest baseline → then the bar.

6 Limitations

- Data: short tasks, one task family, one arm; nothing on long horizon.
- One operator ran the arm and labelled the outcomes, not blind.
- Small samples and one training seed in many tests (the n of each test is in the appendix).
- A fail on our rig is not a fail on the authors' hardware.
- Papers chosen from the stack we run (world models, VLAs, failure monitors, chunking, edge latency): the failure rate describes these papers, not the field.
- Our own mistakes are possible: TurboVLA (Section 3).

7 Related work

- On the same arm: a real-world VLA benchmark with a taxonomy of failures and recoveries [13]; ArmnetBench, a fleet of SO-101 cells evaluating policies at scale [14].
- In simulation: vla-eval, VLAs across 14 benchmarks [8]; a benchmark of demonstration quality [21].
- They rank policies; we test one paper claim against a bar fixed before, on one cheap rig.
- Closest: RoboSeed [26], scores VLA and world-action models on five dimensions including reproduction evidence and deployment affordability; its authors say the score is not a probability that a result is correct.
- Ours: one yes or no per claim on one rig, with the data behind it.

8 Conclusion

- 37 claims from 21 papers on a €1,350 rig, $42.30 of cloud GPU → 12 hold.
- Several lost to a baseline with no learning: copy-last, the joints, episode length.
- Advice: run the cheapest baseline first, on the same data, with the bar fixed before the test.
- Everything is public and every number can be rebuilt from the repository.

References

[1] DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning, 2024. arXiv:2411.04983.

[2] Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies, 2025. arXiv:2503.08558.

[3] SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025. arXiv:2506.01844.

[4] Real-Time Execution of Action Chunking Flow Policies, 2025. arXiv:2506.07339.

[5] SAFE: Multitask Failure Detection for Vision-Language-Action Models, 2025. arXiv:2506.09937.

[6] VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model, 2026. arXiv:2602.10098.

[7] Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons, 2026. arXiv:2603.02115.

[8] vla-eval: A Unified Evaluation Harness for Vision-Language-Action Models, 2026. arXiv:2603.13966.

[9] LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels, 2026. arXiv:2603.19312.

[10] Hierarchical Planning with Latent World Models, 2026. arXiv:2604.03208.

[11] Adaptive Action Chunking at Inference-time for Vision-Language-Action Models, 2026. arXiv:2604.04161.

[12] When Does LeJEPA Learn a World Model?, 2026. arXiv:2605.26379.

[13] Benchmarking Vision-Language-Action Models on SO-101: Failure and Recovery Analysis, 2026. arXiv:2606.08881.

[14] ArmnetBench v0.1: Parallel Real-World Evaluation of Manipulation Policies on a Low-Cost Arm Farm, 2026. arXiv:2607.24481.

[15] ContactGuard: Pre-Contact Execution Monitoring with Action-Conditioned Latent World Models, 2026. arXiv:2608.13438.

[16] VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies, 2026. arXiv:2609.06508.

[17] D-JEPA: A Decision-Aligned Latent World Model, 2026. arXiv:2609.24749.

[18] H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning, 2026. arXiv:2610.06805.

[19] RoboJEPA: Scaling Robotic Latent World Models, 2026. arXiv:2610.10515.

[20] ReAP: From Visual Failure Recognition to Action-Conditioned Risk Prediction for Runtime Failure Monitoring, 2026. arXiv:2610.16309.

[21] What Matters in Quality-Aware VLA Pretraining: A Benchmark and Empirical Study, 2026. arXiv:2610.18869.

[22] REKI: Registering Knowledge in VLAs, 2026. arXiv:2610.22178.

[23] PACT-VLA: Speculative Action-Chunk Verification with Pre-Commit Observations, 2026. arXiv:2610.28019.

[24] Coupling Execution Horizon and Sampling Depth in Flow-Matching VLA Policies, 2026. arXiv:2610.32102.

[25] Tracking Task Progress Through Errors and Recovery:A Benchmark and Recurrent Distributional Model, 2026. arXiv:2610.39125.

[26] Xin Ma et al. RoboSeed: Multidimensional Evaluation of Vision-Language-Action and World-Action Models: Capability, Resource Affordability, and Research Evidence, 2026. arXiv:2610.40026.

Appendix: every paper

paper	their rig	claim tested	bar	our result	verdict
LeWorldModel [9]	sim	A small JEPA world model learns the arm's dynamics well enough to plan with it.	Dynamics: beat copy-last, every corruption ≥ 1.3 at $K = 30$. Planning: rank the executed chunk first among 8 in ≥ 0.8 of anchors.	dynamics 0.31 vs 0.89 copy-last (reproduces); planning top-1 0.60 vs 0.68–0.90 for an action-only baseline (fail)	fail
DINO-WM [1]	sim	A predictor on frozen DINOv2 patches plans better than one on a learned latent.	G1b top-1 ≥ 0.8 at some target distance.	arm-pose R^2 0.92 (vs 0.57 for W1); G1b top-1 0.55–0.61; no-model baseline 0.68–0.90	fail
RoboJEPA [19]	>$20k	Pretrained RoboJEPA predicts our latents, uses the actions, and runs on the robot's computer.	9-step L1 ≥ 15 % below copy-last (300M) and ≥ 5 % below shuffled actions; 2-view encoder ≤ 200 ms on the Orin.	11.8 % below copy-last (300M; 1B 15.8 % on 30 clips); 9–10 % below shuffled; encoder 526 ms on the Orin NX	fail
H-JEPA [18]	sim	A slow second level makes latent distance track task progress.	Progress monotonicity Spearman $\rho \leq -0.80$; false stalls ≤ 10 %.	level-2 ρ -0.73; false stalls 52 %	fail
D-JEPA [17]	<$5k	A decision-aligned ranking operator on a frozen world model picks better action chunks.	G1b regret ≤ 0.10 and $\leq$ the action-only baseline + 0.05.	regret 0.263 (W1 alone 0.263; action-only -0.007)	not tested
ContactGuard [15]	$5–20k	An event-anchored probe on imagined latents predicts grasp failure better than the current latent.	AUROC ≥ 0.85 and ≥ 0.05 above the current latent.	0.70 imagined vs 0.82 current	fail
FAIL-Detect and SAFE [2] [5]	>$20k	A conformal band on a learned failure score keeps false alarms at α while catching failures.	Successes flagged $\leq 1/20$; failures flagged ≥ 50 %.	22.7 % of successes flagged at $\alpha = 5$ %; 46 % of failures caught	fail
ReAP [20]	$5–20k	A risk head on an action-conditioned latent warns before failures.	$\geq 5/8$ failures caught at $\leq 1/20$ false interventions; action-conditioned iff true $\geq$ shuffled + 0.05.	v1 AUROC 0.93 came from time and length; v2 0.789 vs 0.708 shuffled, 3/8 at 5/42	fail
VLA-Corrector [16]	sim	A verifier on observable history flags a failing rollout early enough to correct it.	$\geq 5/8$ failures at $\leq 1/20$ false interventions.	the paper's verifier family on our rollouts (a GRU on pooled RGB statistics and actions): 2/8 caught at 3/42 false interventions, AUROC 0.59; cosine drift of the W1 latent 0/8, AUROC 0.30	fail
Robometer [7]	>$20k	A 4B reward model judges SO-101 rollouts zero-shot.	Pooled AUROC ≥ 0.85 and within-policy ≥ 0.8 on ≥ 5 of 7 policies.	pooled AUROC 0.818 on 2099 ArmnetBench rollouts; episode length alone 0.938	fail
SmolVLA [3]	<$5k	A 450M VLA succeeds on an SO-101 task and can run on low-cost hardware.	10-trial operator-judged gate (no numeric pass mark registered); chunk inference within the async budget (667 ms).	9/10 on bowls; 974 ms in PyTorch, 238 ms with vla.cpp on the Orin NX	reproduces

Table 3: One row per paper with a claim of its own (internal model and test names kept). The rig price of €1,350 counts the parts with a stated price; 2 small parts are not priced yet. Read the verdict against the rig each paper was shown on.

paper	their rig	claim tested	bar	our result	verdict
VLA-JEPA [6]	sim	VLA-JEPA trains cheaply on SO-101 data and runs on the robot.	10-trial operator-judged gate (no numeric pass mark registered); for reranking (H0): oracle / mean error $\leq$ 0.7 and N = 8 $\leq$ 1.5$\times$ the latency.	9/10 for $9.7; 355 ms per chunk on the Orin NX; reranking no-go (0.76, 1.84$\times$)	reproduces
REKI [22]	sim	Policies without memory oscillate between poses at a measurable rate.	$\geq$ 10 % of failed episodes oscillate (their criterion).	20.3 % of all episodes, 24.0 % of failures; REKI trained on my data: 19/20 + 19/20 on the arm, oscillates in 1/40	reproduces
Real-Time Chunking [4]	unstated	Conditioning a new chunk on the executing one removes the jump at chunk boundaries.	Best gradient-free method cuts the median jump $\geq$ 50 % against naive async, drift $\leq$ +10 %, within 667 ms.	prefix overwrite cuts the jump 71–73 % against naive async at +0.4 ms; tuned LeRobot identity mode is close	not proven
HWM (hierarchical latent world models) [10]	>$20k	A high-level world model over learned macro-actions beats the low-level model beyond about 1 s (their Fig. 6 crossover).	Crossover in [1.0, 2.0] s; at 2 s high-level error $\leq$ 0.95 $\times$ low-level; learned macro-actions $\geq$ net joint displacement on cosine.	no crossover: W1 0.382 $\pm$ 0.005 $\rightarrow$ 0.563 $\pm$ 0.007, high level 0.769 $\pm$ 0.009 $\rightarrow$ 0.759 $\pm$ 0.009 (0.73–2 s); cosine 0.608 $\pm$ 0.007 vs 0.551 $\pm$ 0.006	not proven
LeJEPA identifiability [12]	sim	A SIGReg JEPA trained on goal-directed data recovers the true latent state only partially and unevenly.	W1's mean R^2 from latent to joints $<$ 0.80; anisotropy (max $-$ min per-joint R^2) $\geq$ 0.20; the higher-R^2 model has the higher G1b top-1.	W1 mean R^2 0.565, anisotropy 0.639 (shoulder lift 0.91, wrist flex 0.27); frozen DINOv2-S 0.867 / 0.301	reproduces
SURVEYOR	sim	A threshold on the imagined-vs-reached latent distance accepts good plans and rejects bad ones ($\tau = 0.20$).	τ below the 10th percentile of non-equivalent pairs and $\leq$ 0.40; as a monitor $\geq$ 5/8 failures at $\leq$ 2/42; $\geq$ 0.05 AUROC over a shuffled drafter.	τ = 1.249 on W1 latents of 69 real rollouts; as a monitor 3/8 at 6/42 (S = 30), AUROC 0.756 vs shuffled 0.560	fail
Adaptive Action Chunking (AAC) [11]	>$20k	Chunk length chosen from the entropy of N sampled chunks follows the task's phases (long in transport, short at the grasp), lowers the boundary jump, and costs little with batched sampling.	median h* in transport $\geq$ grasp window + 2; jump with AAC + overwrite $\leq$ fixed + overwrite at d = 8; an N = 20 decision $\leq$ 298 ms on the Orin.	h* 2 in transport and 2 at the grasp (86 % of decisions at the floor); jump 0.973 vs 0.940; N = 20 decision 4,751 ms (sequential) on the Orin	fail
PACT-VLA [23]	$5–20k	Picking among N sampled chunks by their flow residual under the pre-commit observation fits what happens next better than a random or stale pick, with no added control-step delay.	PACT beats a random pick and the stale-observation residual at d = 8 (CIs $<$ 0); its delay on the Orin $\leq$ the naive delay; at that delay it still beats a single sample.	offline $-0.27°$ vs random and vs the stale residual (both CIs $<$ 0); on the Orin 1,200 ms per cycle, 37 control ticks vs 8	fail
Horizon $\times$ depth (Euler steps vs execution horizon) [24]	>$20k	With long open-loop commitment, cutting a flow-matching VLA's Euler steps from 10 to 2 contracts the action cloud around an unchanged centroid, saves most of the inference time and avoids edge-GPU thermal throttling.	median R(2)/R(10) $\leq$ 0.70 with the centroid within 0.05 relative and 3°; N = 2 latency $\leq$ 0.85 $\times$ N = 10 on the Orin, N = 10 within 5 % of 238 ms.	R(2)/R(10) 0.78 (CI 0.73–0.81), centroid 0.018 and 2.35°; 196.1 vs 237.5 ms per chunk (0.83, vision 151 ms); no throttling at N = 10 on the Orin, at MAXN_SUPER or 25 W	not proven
REMORA [25]	>$20k	Reading a progress distribution out at the supported peak nearest the previous estimate steadies the estimate and lets it fall during real setbacks, with no worse detection of the stages.	Against per-frame argmax: premature not higher and missed $\leq$ +2 points (P1); flips halved, CI upper $\leq$ 0.7 (P2); a decrease inside $\geq$ 50 % of the reversal windows with $\leq$ 20 % false regressions (P3).	premature 0.22 vs 0.25, missed equal (P1 pass); flips 0.70 $\times$ argmax, CI 0.48–0.94 (P2 fail); setback recall 2/7, false 0.52 (P3 fail); MAE 0.184 vs median 0.218	fail

Table 3, continued.